

真实世界中的 KVCache 缓存： 某大型云服务商的特征刻画与优化

Jiahao Wang^{†1}, Jinbo Han^{†1}, Xingda Wei^{*1}, Sijie Shen², Dingyan Zhang¹, Chenguang Fang², Rong Chen¹,
Wenyuan Yu², and Haibo Chen¹

¹ 上海交通大学并行与分布式系统研究所

² 阿里巴巴集团

摘要

云服务商迫切需要高效地服务大语言模型 (LLM)；在处理每个请求后缓存其中间结果 (KV\$)，能够显著改善服务吞吐量与延迟。然而，人们对于 LLM 服务究竟如何受益于 KV\$ 缓存仍缺乏充分认识，而缓存淘汰策略等系统设计决策又高度依赖工作负载。

本文首次对一家领先 LLM 服务商的 KV\$ 工作负载模式进行了系统刻画。相较于聚焦合成工作负载的既有研究，我们得到了一些此前未被覆盖的观察：KV\$ 的复用在请求之间呈偏斜分布，单轮请求之间的复用与多轮请求同样重要；从所有请求整体看，复用时间与复用概率具有多样性，但对某一具体请求类别而言，其模式往往可预测；为达到理想缓存命中率所需的总体缓存容量较为适中。基于这些特征，我们进一步提出一种工作负载感知的缓存淘汰策略；在真实轨迹下，尤其当缓存容量受限时，该策略能够改善服务性能。

1 引言

大语言模型 (LLM) 凭借在多种任务上涌现出的能力，已经成为重要的云服务。面向消费者的服务 (to-C)，如基于对话的聊天机器人 [42]，以及通过 LLM API 调用提供的企业服务 (to-B)，都已广泛部署在云端 [41, 20, 4]。作为在线服务，以低延迟、高吞吐量服务 LLM 至关重要；但由于每个 LLM 请求都需要巨量计算，要以成本高效的方式实现这一目标并不容易。

缓存已服务请求的中间结果 (KV\$) 已成为改善服务性能的常见方法 [44, 5]：当两个请求共享相同输入前缀时，它们的 KV\$ 相同；因此，若缓存第一个请求的 KV\$ (即 KV\$ 缓存)，便无需为后续请求重复计算。命中时计算量减少，从而降低服务延迟并提高吞吐量。

与传统数据缓存系统 [59, 7, 39] 类似，缓存淘汰策略等 KV\$ 缓存系统组件的有效性，应针对工作负载的 KV\$ 复用特征进行定制。然而，LLM 服务工作负载包含多样的请求类型，而且每种请求类型又具有独特特征，因而十分复杂。例如，生产级 LLM 服务既要处理 to-C 工作负载 (如聊天)，也要处理 to-B 工作负载 (如通过 API 调用实现任务自动化)。这些请求还可能以单轮或多轮交互方式处理 (第 2 节)。由于 KV\$ 可跨不同单轮请求与多轮请求复用，

	时间	类型	UID	轮次	内容
ShareGPT [2]	✗	✗	✗	✓	✓
Mooncake [1]	✓	✗	✗	✗	✓
本文轨迹	✓	✓	✓	✓	✓

表 1: 目前可用的 LLM 服务轨迹对比。时间指每个请求的调用时间；类型指请求类别，例如是否为 API 调用；UID 指发起请求的用户 ID；轮次指请求的多轮信息；内容指每个请求 (匿名化后) 的输入与输出 token。

单轮请求类型与多轮交互模式都可能影响 KV\$ 缓存的设计选择。

由于缺乏对真实轨迹的深入分析，人们对常见 LLM 服务工作负载中的 KV\$ 缓存仍存在显著认知缺口。首先，生产工作负载是否存在大量复用、哪类请求贡献最多复用，尚不明确。其次，KV\$ 的复用时间或复用概率分布仍然未知，而这些分布对缓存策略设计至关重要。最后，现有工作缺少对 KV\$ 生命周期的刻画，而该信息对于估计真实 LLM 服务所需的缓存容量不可或缺。

刻画生产轨迹。为弥补上述缺口，我们从世界上最大的云服务商之一 (阿里云) 收集并刻画了两类具有代表性的生产服务工作负载：其一是 to-C 工作负载，用户通过浏览器聊天机器人或移动应用向云托管 LLM 服务提交请求；其二是 to-B 工作负载，开发者在程序中通过 OpenAI 兼容接口向 LLM 服务发送 API 请求。在公司严格隐私政策允许的范围内，我们尽可能收集了详细信息，包括请求提交时间、请求类型 (单轮、多轮、API 请求或其他) 以及用于分析缓存有效性的匿名化内容。据我们所知，现有其他 LLM 服务轨迹均无法同时达到本文轨迹的信息深度与规模 (见表 1)。例如，ShareGPT [2] 仅提供各请求的输入和输出，而不提供请求提交时间；后者对分析真实 KV\$ 命中率及其随时间的使用情况至关重要。我们相信，所收集的工作负载代表了典型 LLM 服务工作负载，并能为今后的 KV\$ 缓存系统设计奠定基础。

从同时提高性能与资源利用效率的 KV\$ 缓存系统设计视角看，本文的关键结论如下：

1. KV\$ 复用十分常见，但其复用比例低于合成数据集研究中报告的数值 [18, 64]。复用呈偏斜分布：10% 的

[†] 两位作者对本工作贡献相同。* Xingda Wei 为通讯作者 (wxdwfc@sju.edu.cn)。

KV\$ 块贡献了 77% 的复用。不同轨迹中的复用来源也不相同：与“多轮请求可能主导大部分复用”的直观认识相反，在 to-B 工作负载中，单轮请求贡献了 97% 的 KV\$ 复用（第 3.2 节）。

2. 与 KV\$ 设计相关的关键工作负载特征，例如复用时间与复用概率分布，会随 LLM 请求类型（API 或聊天）以及多轮请求的轮次而变化。尽管存在这种多样性，但对每个具体请求类别（类型与轮次的组合），都可以根据历史信息预测复用时间（第 3.4 节）。服务系统已知请求类别，因而可系统地利用估计概率改进缓存策略。

3. KV\$ 的生命周期短暂：to-B 工作负载中 KV\$ 生命周期的 P99 为 97 秒，因此通常只需较小缓存。例如，对使用常见 GQA [3] 的模型，在采用 LRU 等标准缓存淘汰策略时，to-B 工作负载中每块 GPU 配置相当于 2 倍 GPU HBM 的 KV\$ 容量，即可接近无限容量下的理想命中率（第 3.5 节）。

上述刻画揭示了当前 KV\$ 缓存系统可能忽视的若干重要设计决策。第一，优化单轮请求缓存与优化多轮请求同等重要。第二，相比仅采用 LRU 这类工作负载无关策略，采用工作负载感知的缓存策略可以提高命中率。第三，KV\$ 生命周期短暂意味着最不经使用（LFU）策略并不合适，因为一个短命 KV\$ 在过去也可能有很高的访问频率。与此同时，对于 API 主导的 to-B 工作负载，小型（GPU 上）缓存已经足够，从而可以免去部署和管理 CPU-RDMA-SSD KV\$ 存储层次的成本与复杂性。除上述主要结论外，我们还发现了一些有趣现象，例如尽管用户可能通过提示词库 [6] 共享提示词，跨用户的 KV\$ 命中仍然极低（第 3.2 节）。

改进的 KV\$ 缓存淘汰策略。基于这些发现，我们提出了一种工作负载感知的 KV\$ 淘汰策略（第 4.2 节），利用每类工作负载经分析得到的复用概率分布，提高真实工作负载下的缓存命中率。具体而言，我们使用按经验采样的各工作负载复用概率分布来确定每个 KV\$ 块的优先级，同时考虑刻画出的空间局部性与 KV\$ 生命周期。为验证改进设计的有效性，我们将策略集成到云上流行的 LLM 服务系统 vLLM [29] 中。基于生产轨迹的大量评估表明，与 LRU、LFU 等工作负载无关策略相比，该设计可将缓存命中率提高 3.9%，并将平均响应时间最多改善 41.4%。

讨论。继续之前，我们说明本工作的两个局限。第一，结果基于大规模 LLM 服务工作负载为期一周的生产轨迹。虽然我们认为聊天机器人和 API 工作负载代表了当前典型情形，但 LLM 服务工作负载仍在演化，推理 [43] 等新工作负载正在出现；对这些新工作负载的刻画留作未来工作。第二，我们主要关注 KV\$ 缓存策略设计，但认为全局调度策略等其他 LLM 服务系统组件也可能受益于这些发现；其他探索同样留作未来工作。尽管如此，我们相信“先刻画真实服务特征、再优化具体服务组件”的方法能够推广到其他 LLM 服务工作负载与系统。

本文所分析轨迹的样例见 <https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon>，淘汰策略的参考实现见 <https://github.com/vllm-project/vllm/pull/22236>。

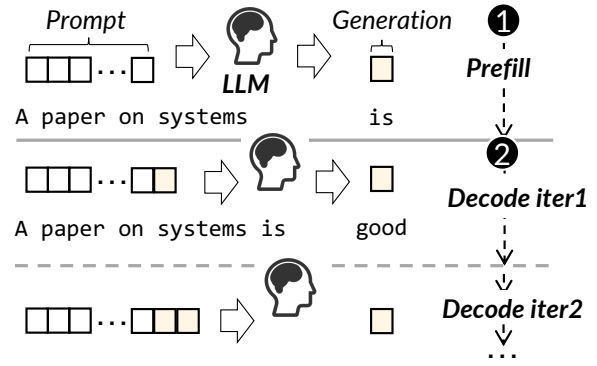


图 1: LLM 处理请求过程示意图。

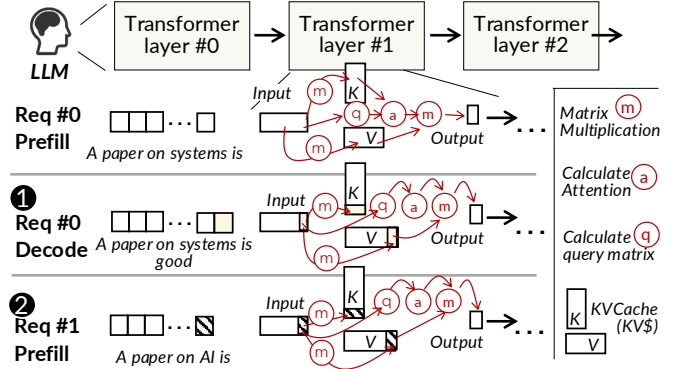


图 2: 示意图：① 预填充请求 (Req#0) 产生的 KV\$ 如何被 Req#0 的解码阶段复用；② KV\$ 如何被未来请求 (Req#1) 的预填充阶段复用。

2 背景：LLM 服务、KVCache、KVCache 缓存与多轮服务

LLM 服务。如图 1 所示，大语言模型 (LLM) 以自回归方式处理请求。给定一个请求（或一批请求），其中每个请求都包含多个 token（即提示词），模型首先执行预填充阶段，通过一次前向传播生成第一个 token (①)。随后模型进入解码阶段 (②)，通过多次前向传播迭代生成余下 token；当模型输出序列结束 (EOS) token 时，迭代终止。在每次迭代中，输入既包含原始提示词，也包含此前生成的所有 token。

KVCache (KV\$)。如图 2 所示，LLM 的一次前向传播包含三个步骤 [54]：(1) 通过矩阵乘法计算 Q、K、V 矩阵；(2) 利用 Q 与 K 得到注意力分数；(3) 将注意力分数与 V 矩阵结合，产生注意力结果。这些步骤计算密集。由于具有相同 token 的输入共享相同 K、V 矩阵（见①），所有 LLM 服务系统都会在 GPU 上缓存预填充阶段产生的 K、V 矩阵，以节省计算时间；随后同一请求在解码阶段复用这些缓存的 K、V 矩阵，避免重复计算，因而它们被称为 KV\$。

KV\$ 缓存与 KV\$ 块。除处理单个请求时在预填充和解码阶段之间复用 KV\$ 外，只要不同请求共享相同 token 前缀，KV\$ 也可跨请求复用。如图 2 中的②所示，“A paper on”的 K、V 值在 Req#0 与 Req#1 中完全相同。因此，即使 Req#0 已完成解码，仍可保留其 KV\$ 供 Req#1 使用，从而显著改善 Req#1 的响应时间（也称首 token 时间，TTFT）：其预填充阶段只需计算“AI is”的 K、V。由于节省了计算，KV\$


```
{ "timestamp": "202x-xx-yy 17:41:33.945",
  "chat_id": "1234567890abcdef1234567890abcdef",
  "parent_chat_id": "aabbccddeeff00112233445566778899",
  "user_id": "1000001234567890",
  "req_type": "text",
  "num_input_tokens": 11026,
  "num_output_tokens": 312,
  "tokens_hashes_input": [1001,2002,3003,4004,5005,...],
  "tokens_hashes_output": [1234,2341,3412,4123,10000,...], }
```

图 3: 所收集轨迹记录的一个示例。

缓存还会提高服务吞吐量。基于这些原因, 现有服务系统都会缓存已完成请求的 KV\$。本文称这种方法为 KV\$ 缓存 [29, 18, 19, 68, 25, 36, 64], 尽管它也常被称为前缀缓存或提示词缓存。KV\$ 可以缓存在 GPU 内存、主机内存 (或远程机器内存), 甚至 SSD 中 [29, 18, 64]。

逐 token 检查前缀匹配可能代价很高, 因此现有 KV\$ 系统以块为缓存粒度, 每个块包含可配置数量 tokens 的 KV。例如, vLLM [29] 默认将 16 个 token 组成一个块。

多轮服务。通过多轮对话与 LLM 交互, 是改善服务质量的一种常见模式 [57, 55]。具体而言, 收到 LLM 的生成结果后, 用户可能使用新提示词发起下一轮请求。由于 LLM 需要此前的对话上下文, 服务系统会将先前各轮请求的输入和输出拼接到新请求输入中, 再将新输入送入 LLM。为帮助管理多轮请求历史, 同一多轮对话中的请求通常归为一个会话。由于当前 LLM 系统会有意把历史输入输出附加为每个新请求的前缀, 多轮请求自然会复用先前轮次的 KV\$。

3 刻画真实世界中的 KV\$ 工作负载

3.1 轨迹数据收集

我们从阿里云通义的一个服务集群收集并分析了两周 (2025 年 2 月和 2024 年 12 月) 的 LLM 服务请求轨迹。为简洁起见, 每条轨迹各展示一个代表日的结果; 在每周的工作日中, 结果始终保持一致 (见第 A.1 节)。需要注意: 受阿里云严格隐私政策限制, 我们无法收集每个请求的原始输入和输出。对于某一条轨迹, 其中收集的所有请求都由同一个模型服务。

下面先说明服务集群运维团队提供的匿名轨迹记录细节, 再突出两条轨迹的差异; 二者均代表当前典型的服务工作负载。图 3 展示了一条所收集请求的记录布局:

1. 时间戳: 服务集群接收请求的时间。
2. 对话 ID: 请求的唯一 ID。
3. 父对话 ID (多轮服务中): 对于多轮会话中的请求, 服务系统会为每个请求分配父 ID, 将其与会话中的前一个请求关联。沿父 ID 回溯, 即可识别属于同一会话的所有请求。若父 ID 为空, 则该请求是会话的第一轮, 本文称之为单轮请求; 否则, 将父 ID 非空的请求称为多轮请求。
4. 用户 ID: 提交请求的用户唯一 ID。为实现匿名化, 这些 ID 被映射到随机选择的域。
5. 请求类型: 服务网关将 LLM 请求分成五类: Text, 通过

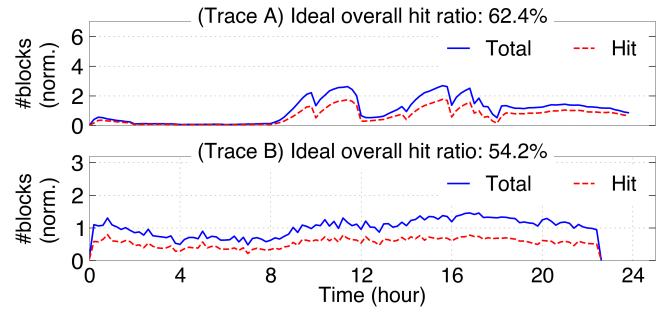


图 4: 一天内真实 LLM 服务工作负载中, KV\$ 缓存的理想命中率分析。访问块数与命中块数均经过归一化。

ChatBot 等界面输入纯文本; File, 文件分析 (例如用 LLM 总结文档内容); Multimodal, 通过 OCR 增强处理进行图像理解; Search, 前端集成网页搜索以增强知识; API, 通过 OpenAI 兼容接口进行程序化访问 [41]。

6. 输入/输出 token 数: 提供给 LLM 的输入 token 数量, 以及模型生成的输出 token 数量。

7. 输入/输出 token 的哈希值: 受严格隐私政策约束, 运行时系统通过带盐哈希和域重映射生成轨迹。具体而言, 收集器使用 SipHash [8], 以每条轨迹随机生成的盐为每连续四个 token 生成唯一哈希值, 再把哈希 ID 重映射为连续自然数。与单 token 哈希相比, 这种细粒度哈希既支持更细致的缓存分析, 也增强了隐私保护。

轨迹 A 与轨迹 B。我们收集了代表两类典型 LLM 服务工作负载的轨迹 A 和 B。轨迹 A 表示常见的 to-C (消费者) 场景, 用户通过云平台构建的服务与模型交互, 即 ChatBot (Text) 或 File、Multimodal、Search 服务。to-C 工作负载的一个关键特征是通常有人参与闭环。轨迹 B 表示 to-B (企业) 工作负载, 企业用户在计算机程序中调用云平台托管的 OpenAI 兼容 API 与 LLM 交互 [41]。API 调用对企业用户十分重要, 因为它使用户能够系统地利用 LLM 自动完成文档翻译、用户请求分类等任务。轨迹 B 不包含请求类型信息, 因为与当前其他厂商提供的 LLM 接口 [41, 20, 4] 类似, 这些信息在客户端被拼接到请求中。

3.2 真实世界中的 KV\$ 复用分析

有多少 KV\$ 可以复用? 图 4 展示了两条轨迹在缓存容量无限假设下的理想命中率。LLM 工作负载确实可以从缓存中受益: 轨迹 A 与 B 的命中率分别为 62% 和 54%。命中率按如下方式计算: 若一条轨迹中 LLM 原本需要计算的 KV\$ 块总数为 N , 命中率为 62% 意味着其中 62% 的块可以复用先前请求计算并缓存的 KV\$。尽管理想命中率很高, 它仍低于合成工作负载中报告的命中率 (例如超过 80% [64, 68, 18])。

单轮请求与多轮请求对 KV\$ 复用同等重要。人们通常认为, 多轮请求是缓存命中的主要贡献者 [64, 18], 但对 to-B 工作负载 (轨迹 B) 并非如此。图 5 展示了各请求类型的占比及其多轮比例。轨迹 A 以聊天机器人发送的请求为主 (78%), 而除 Multimodal 外, 各请求类型的多轮比例相近 (47–51%)。轨迹 B 仅包含 API 请求, 多轮比例可以忽略不计 (低于 0.1%); 即便如此, 其中单轮请求仍贡献了总缓存命中的 97%。

Types (Trace A)	Workload ratio (%)	Multi-turn Ratio (%)
Text	78	47
File understanding	4	51
Multimodal understanding	3	10
Search	15	49

Types (Trace B)	Ratio (%)	Multi-turn Ratio (%)
API	100	0.08

图 5: 所收集轨迹中的工作负载类型与请求多轮比例。

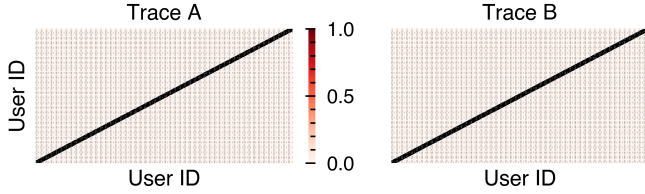


图 6: 一个用户能否命中另一个用户缓存的 KV\$。颜色越深, 归一化命中次数越高。由于轨迹提供方服务的用户很多, 为避免对角线过细, 图中对其进行了放大。

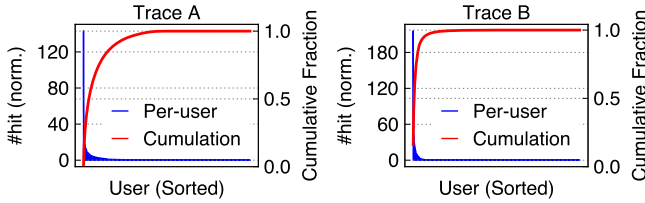


图 7: 无限缓存的理想设置下, 不同用户贡献的归一化命中次数。

单轮 API 请求在 to-B 工作负载中主导命中有两个原因。第一, LLM API 请求通过公共系统提示词共享 KV\$。具体而言, 每个请求由两部分构成: (1) 给出任务指令的系统提示词 (例如文档翻译规范); (2) 包含请求特定数据用户提示词 (例如目标文档)。程序发起 API 调用时 [34], 通常会在程序中硬编码相同的系统提示词, 因此即使请求均为单轮, 也能跨请求共享 KV\$。第二, API 请求的提交速率往往很高 (例如 > 10 QPS), 使同一系统提示词对应的共享 KV\$ 得以频繁复用。

跨用户 KV\$ 命中十分罕见。图 6 在无限容量 KV\$ 缓存假设下, 展示了用户之间缓存命中的热力图; 颜色越深表示归一化命中次数越多。可以看到, 对每个用户而言, 大多数命中来自自己提交的请求 (即对角线)。用户自然会复用自身请求中的 KV\$, 既可能通过多轮对话, 也可能通过程序针对同一任务、使用相同系统提示词调用 API; 而极低的用户间命中率表明, 用户更倾向于自定义系统提示词, 而不是采用第三方提示词库 [6] 提供的模板。因此, 在设计 KV\$ 系统时, 本文建议更关注用户内 KV\$ 复用, 而非用户间复用 [19, 68, 25, 36]。

KV\$ 复用呈偏斜分布。图 7 展示了不同用户贡献的 KV\$ 缓存命中及其累计值。命中模式高度偏斜: 轨迹 A 中 19% 的用户请求、轨迹 B 中 4% 的用户请求, 贡献了超过 90% 的 KV\$ 块命中。偏斜来自两个因素。第一, 请求在用户之间分布不均, 轨迹 B 尤为明显。图 8 展示了不同用户发

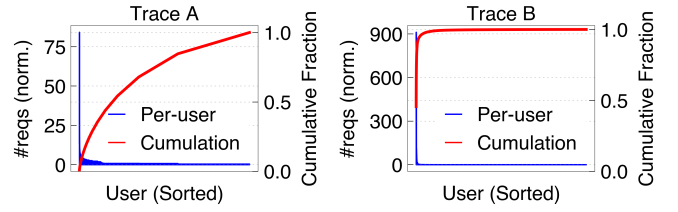


图 8: 不同用户的归一化请求数。

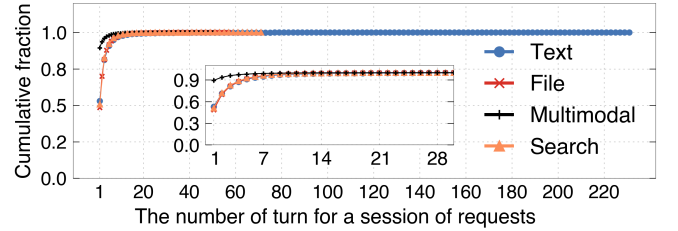


图 9: 轨迹 A 中多轮请求的轮次数分布。

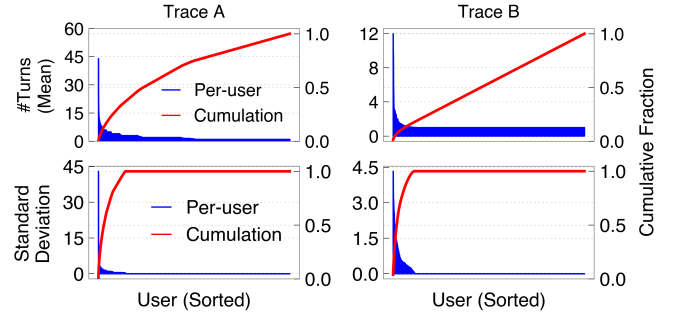


图 10: 每个用户所发送请求的平均轮数 (上) 与标准差 (下)。用户分别按平均轮数 (上) 和标准差 (下) 排序。

送请求数的归一化值: 轨迹 B 中 15% 的用户 (头部用户) 贡献了 90% 的请求。由于 KV\$ 命中主要发生在同一用户发出的请求之间 (见图 6), 请求数的偏斜自然会导致类似的命中偏斜。第二, 即使不同用户发送的请求相对均匀, 各用户的 KV\$ 命中数仍有显著差异。如图 8 所示, 轨迹 A 的请求分布偏斜程度适中: 19% 的用户产生 50% 的请求, 72% 的用户产生 90% 的请求; 尽管如此, KV\$ 命中依然偏斜。作者推测, 这源自用户参与多轮请求的倾向不同。

3.3 多轮请求特征分析

首先考察多轮请求的特征, 因为它们仍然主导 to-C 工作负载中的 KV\$ 复用。轨迹 B 中多轮请求十分罕见, 因此不报告其对应结果。

多轮请求具有高度可变性。我们观察到两类可变性。第一, 不同会话的轮数差异显著。图 9 展示了轨迹 A 中的轮数分布: 54% 的请求是单轮请求, 第 50 与第 90 百分位轮数分别为 1 和 5。该分布具有长尾, 最长会话包含 232 轮, 是中位数的两个数量级 (232 倍)。

第二, 多轮请求分布在用户层面存在显著差异。图 10 表明, 除少量头部用户外, 大多数用户的平均轮数相近; 轮数最高的 10% 用户比总体平均值高 8.9 倍。此外, 轮数的标准差差异很大, 范围从 1 到 44。这表明用户交互模式多样: 有些用户的轮数变化很大, 另一些则保持稳定。

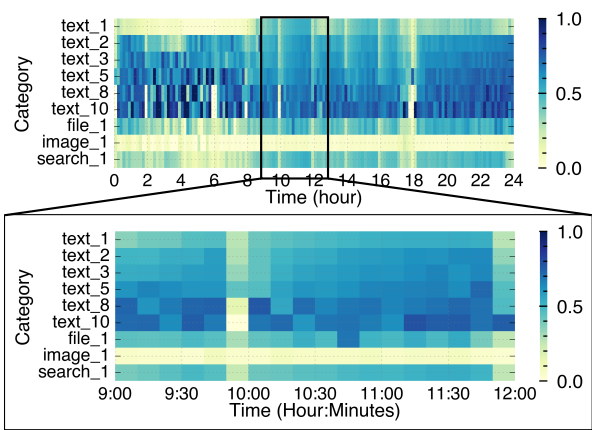


图 11: 不同请求类型发起下一轮请求的频率如何随时间变化。横轴为时间线, 工作负载类别按流行度排序。

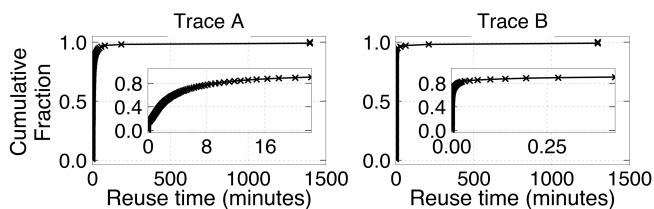


图 12: 轨迹 A 和 B 中所有请求的 KV\$ 复用时间分布。

给定工作负载类别和时间区间后, 发起下一轮请求的概率可预测, 而且不同请求类别的概率不同。请求类别同时包含原始请求类型 (例如 Text、Search) 和请求轮次, 即 1 轮 Text (单轮文本请求) 与 2 轮 Text 被视为不同类别。图 11 以热力图展示了发起下一轮请求的概率在长时间区间 (24 小时) 内的变化, 以及短时间区间 (3 小时) 的局部放大; 颜色强度表示一个时间窗口 (10 分钟) 内分析得到的频率。可以看到, 虽然不同请求类别的频率不同 (表现为竖向色带), 但在某一具体类别内, 发起下一轮请求的频率在一小时区间内保持稳定 (表现为横向色带)。作者将其归因于: 给定特定时间区间与请求类别时, KV\$ 复用的概率分布相当固定; 第 3.4 节对此作进一步说明。有趣的是, 不同请求类别发起下一轮请求的频率确有差异。例如, 与多轮请求 (均值为 0.8) 相比, 单轮请求发起下一轮的频率显著更低 (均值为 0.5)。

3.4 时间与空间局部性分析

时间局部性可以通过分析复用时间分布来刻画; 复用时间是一个 KV\$ 块被另一请求复用时的时间间隔。

KV\$ 复用时间很短。图 12 展示了两条轨迹的 KV\$ 复用时间分布。两种工作负载的复用时间通常都很短: 轨迹 A 中 80% 的复用时间小于 10 分钟, 轨迹 B 中则小于 10 秒。二者差异来自生成请求的交互方式不同, 分别为人在闭环与计算机在闭环。

不同工作负载类型具有不同的时间局部性。第一, 在 to-C 工作负载中, 单轮请求的局部性低得多; to-B 工作负载则相反。图 13 中间的子图报告了两条轨迹中一个单轮请求能否被另一单轮请求复用。可以看到, 轨迹 A 中可复用的单轮请求少于 30%, 而轨迹 B 中超过 50% 的请求可以复

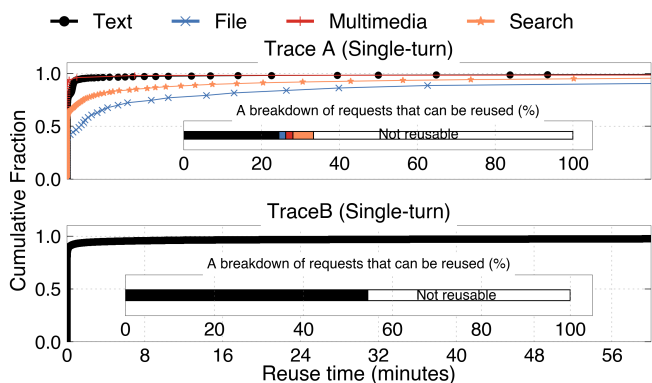


图 13: 轨迹 A 和 B 中单轮请求的 KV\$ 复用时间分布; 轨迹 B 只有一种请求类别 (API)。

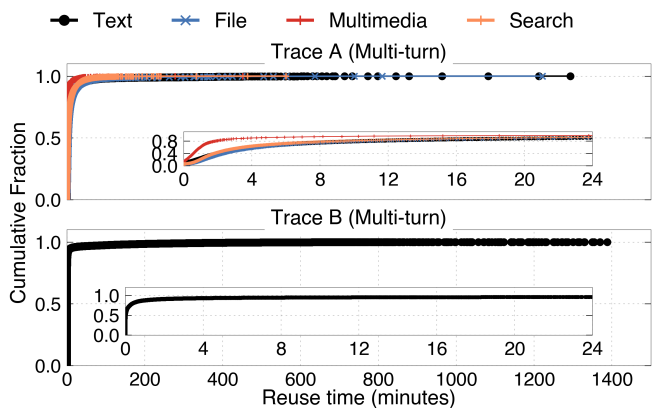


图 14: 轨迹 A 和 B 中多轮请求的 KV\$ 复用时间分布; 轨迹 B 只有一种请求类别 (API)。

用。

第二, 不同请求类型呈现不同的时间局部性。图 14 按请求类型展示多轮请求的复用时间分布。聊天请求 (Text) 的复用时间长于图像理解 (Multimodal), 但短于文件理解 (File)。这源自工作负载自身特征: 与处理日常对话输出相比, 用户通常需要更多时间处理基于文件内容生成的复杂 LLM 输出; 反之, 视觉信息处理比文本处理更快 [47], 因此图像任务的复用时间更短。最后, Search 工作负载的复用时间第二长, 因为用户常常需要大量时间分析搜索结果。

给定时间段和请求类别后, KV\$ 复用时间的概率分布服从指数分布。图 15 给出了不同请求类别中 KV\$ 块的复用时间概率分布。对每个类别, 我们测量其 KV\$ 在缓存后的特定时间间隔被复用的可能性; 这些分布通过统计请求执行后每一秒发生的复用事件频率绘制。

结果带来三项关键观察。第一, 给定请求类别后, 指数分布能够很好地拟合复用概率。第二, 概率具有工作负载感知性, 即不同请求类别的分布不同, 包括请求类型相同但轮次不同的类别 (例如 text-1 与 text-2); 这意味着工作负载感知的设计需要区分不同请求类别。第三, 流量模式相近的请求具有相似分布。以 text-1 为例, 第一行中上午 9:00–10:00 得到的分布与 10:00–11:00 得到的分布相似, 却与夜间 3:00–4:00 得到的分布 (第二行) 明显不同; 不

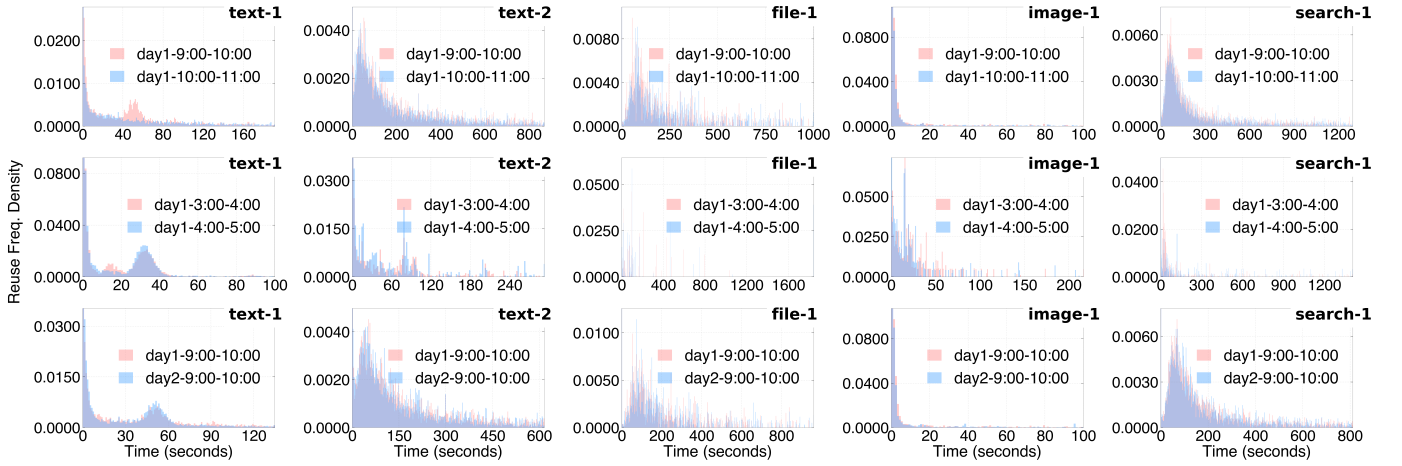


图 15: KV\$ 块复用时间概率分布的经验分析。各列对应不同请求类别；第一行是在高流量（白天）下的分布，第二行是在低流量（夜间）下的分布，第三行比较不同日期但具有相似流量模式时的分布。

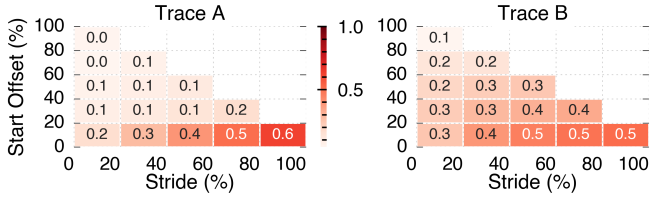


图 16: 两条轨迹的总体空间局部性。

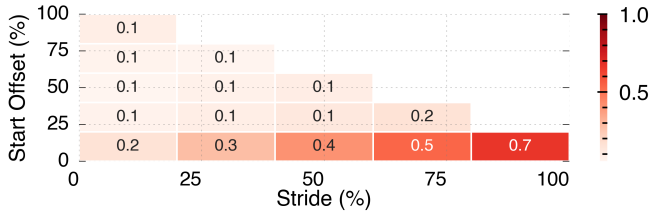


图 17: 轨迹 A 中多轮请求的空间局部性。

同日期之间的分布也相似（第一行与第三行）。第一和第三项观察共同表明，可以使用最近的历史数据获得某一请求类别的分布。

空间局部性。图 16 展示了两条轨迹中 KV\$ 的总体空间局部性。本文用不同缓存步长（即每个请求中被缓存 token 的比例）以及不同缓存起始 token 偏移量下的理想命中率来刻画空间局部性，由此得到二维热力图。例如，偏移量 (10%, 10%) 表示对每个请求，只缓存从 10% token 位置开始的 10% token。

第一，从开头开始缓存时，KV\$ 的空间局部性更好。这符合预期，因为只有前缀完全相同的请求才能共享 KV\$（第 2 节）。本文强调这一看似显然的事实，是因为一些 KV\$ 优化 [26] 会缓存请求尾部 token，通过并行计算与加载隐藏 KV\$ 加载时间；但这种方法可能降低空间局部性。尤其是图 17 表明，对多轮对话而言，复用缓存中的中间部分几乎没有收益。

第二，与时间局部性一样，空间局部性也具有工作负载感知性。在图 18 中，Text 和 Multimodal 等工作负载表现出明显空间局部性，而 File 与 Search 几乎没有空间局部

性。由于无法获得用户输入内容，作者推测原因如下：应用会为 Text 和 Multimodal 工作负载加入固定系统提示词，而 File 与 Search 的系统提示词依赖用户输入。

最后，我们发现，在轨迹 A 中增大步长会提高缓存命中率（图 17），但轨迹 B 并非如此（图 18 最后一列）。轨迹 A 中多轮请求会贡献缓存命中，因此为一个请求缓存更多 token 会带来更多命中。轨迹 B 中只有系统提示词贡献缓存命中，所以缓存请求中 50% 的 token 已达到 50% 的峰值命中率。这意味着轨迹 B 中多数系统提示词长度低于请求总长度的 50%，但作者并无详细请求内容来支持这一推测。

3.5 KV\$ 缓存容量需求分析

确定合适的缓存容量对缓存系统至关重要。不同于传统存储系统，缓存的 KV\$ 生命周期短暂，因此识别恰当的容量需求可以节省平台成本。本节采用自底向上的方式分析 KV\$ 容量：首先刻画请求 KV\$ 生命周期的特征，再分析每个请求的存储用量；两者结合即可估计所需的 KV\$ 总容量。

KV\$ 生命周期短暂且可预测。图 19 分析了 KV\$ 块的生命周期。如果一个 KV\$ 块此后不再被复用，就称其“死亡”。可以看到，大多数 KV\$ 块“存活”时间很短：轨迹 A 中 90% 的 KV\$ 块在 612 秒后不再被复用，轨迹 B 中这一数值为 0.3 秒。这与前文分析一致，例如许多多轮请求的复用时间很短（图 12），而多数请求只有少数几轮（图 9）；一旦多轮请求结束，其 KV\$ 就不再复用。尽管存在少数离群值，轨迹 B 中块的生命周期仍更短。与轨迹 A 不同，轨迹 B 中缓存的多数 KV\$ 块服务于系统提示词，因而在很久以后仍可能被复用。例如，轨迹 B 中最长生命周期为 1,200 分钟，轨迹 A 中为 800 分钟。

图 20 进一步展示了不同请求类别中 KV\$ 块平均生命周期随时间的变化。图中颜色强度表示归一化生命周期，颜色越深表示 KV\$ 块存活越久。尽管平均生命周期在 24 小时时间尺度上随时间变化，但放大到更短时间尺度（上午 9:00–11:00）后，模式相当稳定。这意味着，可以利用历史数据预测 KV\$ 块的生命周期，从而缩减 KV\$ 缓存并节省成本，因为在高速存储介质（例如 CPU DRAM）中存储

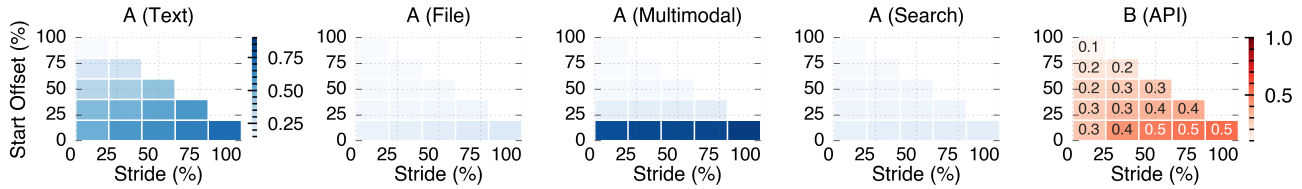


图 18: 单轮请求的空间局部性。

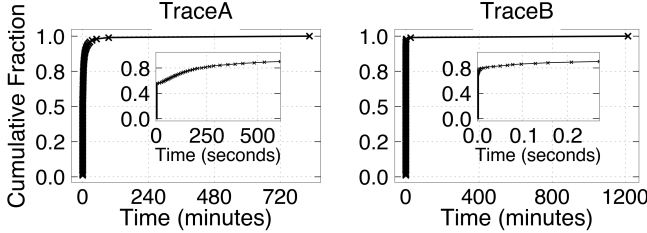


图 19: 两条轨迹中 KV\$ 块生命周期的分布。

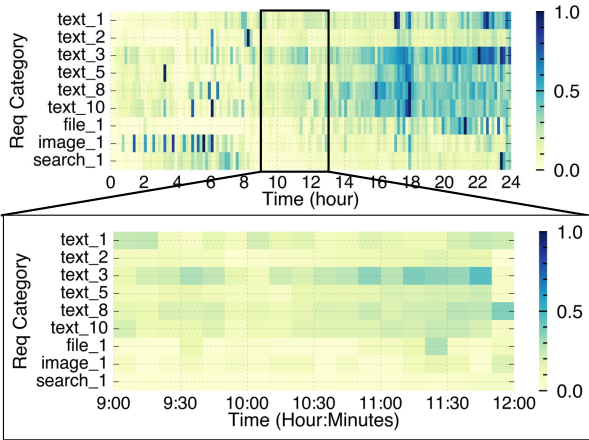


图 20: 不同请求类别中, KV\$ 块的平均生命周期如何随时间变化。横轴为时间线, 请求类别按使用量排序。

KV\$ 的成本很高。

单请求 KV\$ 缓存用量适中。考虑缓存时, 一个关键因素是待缓存 KV\$ 的大小, 它与每个请求的输入长度及相应的模型输出长度成正比。图 21 展示了不同轨迹中的请求大小分布。需要注意, 本文报告的是相对于一个服务实例中可用于服务该请求的 HBM 所归一化的相对大小¹, 因为相对值能更直观地反映缓存每个请求的 KV\$ 所需容量。可用于 KV\$ 的 HBM, 通过从服务实例中 GPU 的总 HBM 扣除模型参数与激活大小得到。由于单请求 KV\$ 大小取决于模型的具体注意力机制, 本文报告了采用两种广泛使用注意力方法的三个代表性模型: 分组查询注意力 (GQA [3]) 与多头注意力 (MHA [54])。GQA 的 KV\$ 内存需求更低。

可以看到, 与 HBM 相比, KV\$ 的总体大小适中。以轨迹 A 为例, 在 Qwen2-7B 等 GQA 模型上, 占主导的 Text 请求类型中, 单轮请求在第 50、90、99 百分位的 HBM 用量分别为 0.09%、0.15%、0.37%, 多轮请求分别为 0.36%、1.31%、2.21%; 单轮与多轮请求的平均用量分别为 0.09%

和 0.56%。用量之所以较小, 是因为每个 token 只消耗数 MB, 而生产轨迹中的请求往往较短: Qwen2-7B 每 16 个 token 消耗 0.875 MB KV\$, 单轮与多轮请求的平均 token 总长度 (输入加输出) 分别为 973 与 5,953。对于其他模型, Llama3-70B 虽然也采用 GQA, 但由于每个实例的 GPU 数量增长较慢, 其 HBM 用量略高: 参数规模增长 10 倍, 而本文测量配置中的 GPU 数仅增长 4 倍。最后, MHA 模型的用量最高, 因为每个 token 的 KV\$ 更大。值得注意的是, 近期开放模型倾向于采用 GQA, 或采用 MLA [35] 等每 token KV\$ 更少的注意力机制。

考虑到每个服务实例通常每秒处理数个请求, 上述刻画意味着实例能够以适中的时长缓存请求。例如, 对 Qwen2-7B, 假设一块 GPU 每秒可处理 5 个请求, 并按每请求平均占用 0.32% HBM 计算 (由 Text 请求 47% 的多轮比例推导), 至少需要 1 分钟才会占满 HBM。该时长落在许多场景 (例如 API) 中 KV\$ 的复用距离内。需要注意的是, 在实际设置中, 由于尾部请求 (例如确实存在上下文超过 12,000 token 的长请求), 可能达不到如此长的复用时长; 同时, 为提高计算利用率, LLM 服务系统通常会以大批次服务请求。

KV\$ 缓存所需的总体内存适中。本文通过测量在理想淘汰策略下, 多大的缓存能够保证理想命中率, 来分析所需的 KV\$ 缓存容量; 理想策略永不淘汰未来还会复用的 KV\$。测量方法如下: 在给定时刻处理一个请求时, 如果该请求的 KV\$ 未来还会复用, 就把其大小加入当前 KV\$ 用量; 同时减去缓存中所有永远不会再复用的 KV\$ 大小。一个棘手之处是, 需求大小取决于为工作负载部署的服务实例数: 若实例过度配置, 估计出的需求会偏小。为得到最大需求, 本文使用最少的服务实例, 即每个实例都以最大 QPS 服务, 从而不存在过度配置。

图 22 给出了相对于服务实例总 HBM 归一化后的结果。GQA 模型只需较小缓存容量即可达到理想命中率: 在轨迹 A 中, Llama3-70B 所需容量仅为可用于 KV\$ 的 HBM 的 4 倍。现代服务集群通常可以满足这一需求, 无需远程内存或 SSD 等更多存储层次 [18]。例如, 在包含 8 块 A100 GPU 和 1 TB CPU 内存的典型设置中, 每块 GPU 平均可使用 128 GB CPU 内存, 约为保留给 KV\$ 的 HBM 空间的 4 倍。此外, 轨迹 B 的 KV\$ 容量甚至小于保留的 HBM, 因此仅在 GPU 上缓存可能已经足够。

尽管 GQA 需要的缓存很小, 仍应注意 MHA 模型确实需要巨量 KV\$ 缓存, 这是因为其每 token KV 对非常大 (见图 21)。因此, 在缓存容量受限时, 优化淘汰策略仍然重要。

¹ 服务实例是服务一个模型所需的最少 GPU 数。例如, 服务 8B 模型只需一块 GPU, 而通常服务 72B 模型需要四块 GPU。

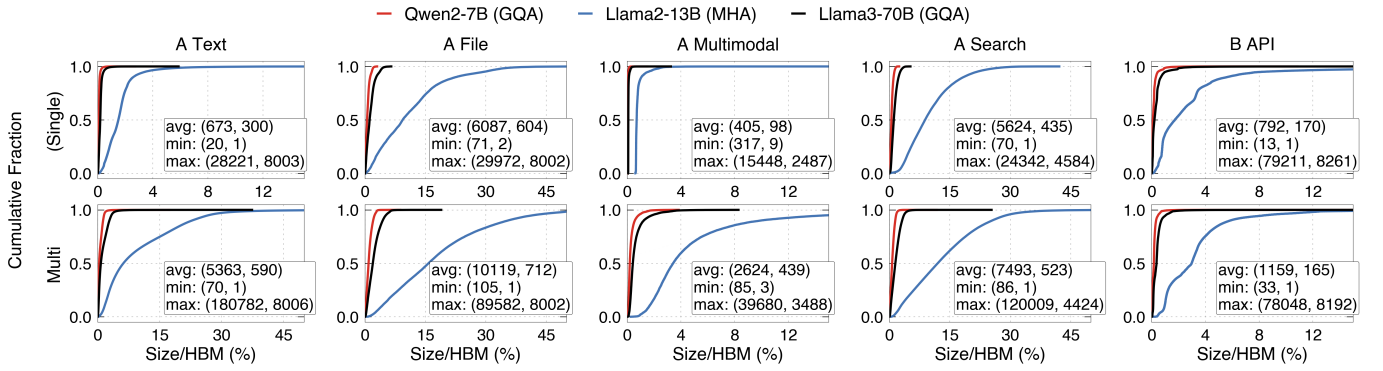


图 21: 不同轨迹中单轮与多轮请求的 KV\$ 大小分布。大小相对于可用于服务的 GPU 内存 (HBM) 进行归一化; 各图元组中的数字表示输入和输出长度。

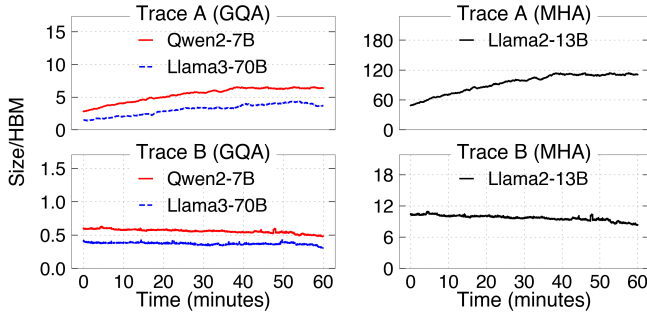


图 22: 不同模型与轨迹中, 达到理想缓存命中率所需的 KV\$ 缓存大小; 数据采集于上午 10:00–11:00。

4 改进的 KV\$ 缓存系统

4.1 设计空间概览

现有 KV\$ 缓存系统 [18, 29, 68, 64] 采用相似的缓存机制与策略。下面先简要概述这些设计, 再介绍本文改进的策略。

当前 KV\$ 缓存机制与策略。在机制方面, 现有系统主要以分层方式在 GPU 和 CPU 上缓存 KV\$。KV\$ 以块为粒度管理 (见第 2 节): 产生一个 KV\$ 块时, 首先将其缓存在 GPU HBM; 如果 GPU 因内存不足而无法缓存新生成的块, 缓存系统会采用下述淘汰策略, 从 GPU 向 CPU 淘汰一些块。被淘汰块按层异步换出到 CPU, 因而换出开销可忽略。如果 CPU 也耗尽内存, 则删除这些块。执行新请求前, 若请求命中缓存块, 服务系统会直接复用, 避免重复计算。若命中位于 CPU, 块也会按层换入 GPU。

在策略方面, KV\$ 缓存系统主要需要两种策略: 如何判断缓存命中, 以及如何淘汰缓存块。对前者, 不同设计检查潜在命中块的粒度不同: 一些系统跨所有请求检查匹配 [68, 25], 另一些仅在单个用户的请求内部检查匹配 [64, 18]。对后者, 当前系统主要采用 LRU 或 FIFO 等标准淘汰策略。尽管它们会加入一些扩展, 例如检查待淘汰块是否仍在调度队列 [18], 总体原理并未改变。

缺点。LRU 等现有工作负载无关策略可能忽略第 3 节分析的工作负载特征, 因而并非最优。以轨迹 A 为例, 在距上次访问经过相同时间 (例如超过 10 秒) 的情况下, 10 轮 Text 请求比 1 轮 Text 请求更可能继续发起下一轮 (见

观察	技术
给定工作负载后, KV\$ 复用概率服从指数分布 (图 11 与图 15)。	① 基于工作负载感知复用概率分布的优先级估计。
空间局部性 (图 16–18)。	② 为前部 KV\$ 块赋予更高优先级。
KV\$ 生命周期短暂 (图 19)。	③ 计算优先级时移除频率, 并限制概率计算的时间跨度。

表 2: 依据第 3 节中的发现, 对工作负载感知缓存淘汰策略主要技术的总结。

图 11)。然而, 如果大量 1 轮请求涌入系统, 由于 10 轮请求的复用时间更长, 它们仍可能被 LRU 淘汰。

本文方案: 基于工作负载感知复用概率分布的 KV\$ 淘汰策略。为此, 我们依据第 3.4 节中的特征刻画, 提出一种考虑各请求类别复用概率分布的工作负载感知缓存淘汰策略。总体而言, 发生淘汰时, 本文会为每个块计算一个优先级, 即依据其请求类别 (工作负载) 经分析得到的复用概率分布, 估计该 KV\$ 未来被复用的可能性。该优先级还综合考虑空间局部性以及 KV\$ 块的短暂生命周期, 优先淘汰优先级最低的块。表 2 总结了本文技术及其在第 3 节中对应的观察。

本文的缓存机制沿用现有系统 [18, 29, 68, 64], 包括异步换出与流水化加载, 因为这些技术已经收敛为标准设计。

4.2 工作负载感知的 KV\$ 缓存淘汰策略

现有工作负载感知策略。工作负载感知的缓存淘汰策略已得到广泛研究 [14, 40, 28, 24, 37, 11, 48]。一个代表性策略是 Greedy-Dual-Frequency-Size (GDFS) [14], 它使用下列与工作负载相关的特征来评估每个缓存对象 (例如 KV\$) 的优先级:

$$\text{Priority} = \text{Clock} + \frac{\text{Frequency} \times \text{Cost}}{\text{Size}} \quad (\text{现有 GDFS}).$$

其中, Clock 捕获对象上次访问时间 (与 LRU 类似), Frequency 是对象累计访问次数 (与 LFU 类似), Size 是缓存对象大小, Cost 是把缓存内容重新带回缓存的代

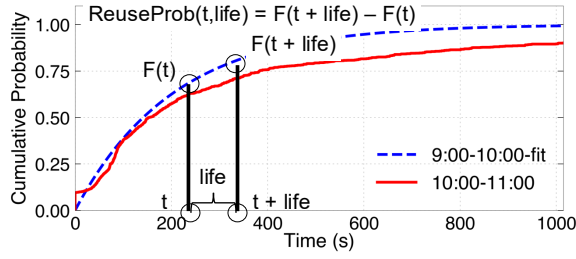


图 23: 给定 KV\$ 块所属工作负载（请求类别）后，计算其复用概率的方法。

价。优先级最低的对象最先被淘汰。

尽管可调整上述 GDFS 公式来计算 KV\$ 块的优先级，但我们发现，它并未充分考虑 LLM 服务中的 KV\$ 使用情况，以及不同请求类别已知的概率分布。例如，由于 KV\$ 块生命周期很短，一个被频繁访问的块未来也可能不再复用，使一个“曾频繁访问但已经死亡”的块浪费 KV\$ 空间。此外，把某个块带回缓存的代价更高，并不意味着其未来更可能被复用；应当优先把更可能复用的块带回缓存。在 GDFS 的设置中无法获得这类工作负载信息，但依据上一节的特征刻画，LLM 服务系统可以直接获得它们。

本文策略。本文沿用 GDFS，为每个 KV\$ 块计算优先级并据此确定淘汰顺序，但用所刻画的 KV\$ 复用特征重新定义优先级：

$Priority = (ReuseProb_w(t, life), -Offset)$ （工作负载感知）。

具体而言，优先级由包含两个指标的元组表示，并按字典序比较。两个指标背后的原理如下：

- **ReuseProb_w** (①) 计算 KV\$ 块在给定时间 t 的复用概率，其中 t 表示距上次访问经过的时间。对每个工作负载（请求类别），复用概率按两步计算。第一，在后台采样最近一段时间内访问的数据。第二，依据图 15 中的观察，用指数分布拟合采样数据，再查拟合曲线得到复用概率。图 23 给出了计算示例：首先使用上午 9:00-10:00 的后台采样数据，拟合给定工作负载复用概率的累积分布函数（CDF，蓝线）。

拟合曲线与真实数据（红线）十分接近。随后即可查曲线，得到一个 KV\$ 块未来的复用概率。该函数还考虑块的预期生命周期（life）；鉴于 KV\$ 生命周期很短，这一点至关重要。

- **Offset** (②) 考虑第 3.4 节描述的空间局部性：优先级与 KV\$ 块在整个请求中的前缀位置成反比。

本文不显式考虑频率 (③)，因为即便对频繁复用的块而言，KV\$ 块的短暂生命周期（见图 19）也使频率无法为缓存策略提供有用信息。更重要的是，还必须用生命周期调节计算出的复用概率。否则，对于复用时间呈长尾分布的请求，拟合概率会在很长时间内返回较高值；这与 KV\$ 生命周期短暂的观察相矛盾，并会导致有限缓存空间利用低效。

详细淘汰算法。图 24 给出了工作负载感知缓存淘汰算法的简化伪代码。发生淘汰时，首先遍历所有缓存块（第 4-6 行），并使用块所属工作负载 ($b.w$) 计算每个块的复用概

Input: B_s : cached KV\$ blocks;
 $Life$: expected lifespan of the KV\$ blocks;
 $H(w)$: hyper parameters for the workload (w)’s fitted probability distribution that are periodically updated in the background.
Output: B : the evicted KV\$ block.

```

1  CurT = time.time()
2  MinPriority = +∞
3  ChosenBlock = Placeholder
4  For  $b$  in  $B_s$ :
5      Priority = ReuseProbH(b.w)(CurT - b.cached_t, Life)
6      ▶ Priority is the reuse probability in
       [CurT - b.cached_t, CurT - b.cached_t + Life)
7      If Priority < MinPriority or (Priority == MinPriority and
        $b.offset > ChosenBlock.offset$ ):
8          MinPriority = Priority
9          ChosenBlock =  $b$ 
10 return ChosenBlock

```

图 24: 本文工作负载感知缓存淘汰算法的简化伪代码。

率；随后将优先级与块的偏移量一起比较，以更新待淘汰块（第 7-9 行）。

性能优化。上述朴素算法的一个缺点是，必须计算所有 KV\$ 块的复用概率，才能找到最不可能复用的候选块。为避免过高计算开销，我们观察到：按照指数分布，每个工作负载内部的所有 KV\$ 块天然可按最后访问时间排序，因此比较同一工作负载中的块时无需计算复用概率。具体而言，对每个工作负载，维护一个按最后访问时间排序的 KV\$ 块优先队列。发生淘汰时，不再扫描所有块，而是先把每个工作负载中最近最少使用的块作为淘汰候选，然后运行图 24 所述算法得到最终淘汰块。这将复杂度从 $O(N)$ 降至 $O(W)$ ，其中 W 是工作负载数量（通常为数十个）。因此，工作负载感知策略的服务性能开销可以忽略：本文观察到每次淘汰的策略延迟为 79 μs ，累计开销仅占典型服务引擎 vLLM [29] 调度开销的 1.2%。

4.3 性能评估

测试平台。评估在一台配有 8 块 NVIDIA A800-80GB GPU 的服务器上进行。GPU 通过 400 GBps 双向 NVLink 互连，因此可以使用多块 GPU（例如 4 块）通过张量并行 [38] 服务一个大模型。GPU 通过 PCIe Gen4 与主机连接，GPU HBM 与主机内存之间的双向带宽最高为 32 GBps。

实现与基线。由于 CachedAttention [18] 和 Pensieve [64] 等同时使用 CPU 与 GPU KV\$ 的先进缓存系统并未开源，本文在默认使用 GPU KV\$ 缓存的先进 LLM 服务系统 vLLM [29] 上实现了 CPU-GPU KV\$ 缓存，并进一步集成了先进系统中的策略与机制（见第 4.1 节）。在相似测试平台和相同合成工作负载下，本文校准确认该实现的性能与 CachedAttention 等先进系统相近。

除在 vLLM 中集成 CPU KV\$ 缓存外，本文还实现了 Mooncake [1] 中的 KV\$ 中心化全局调度器，因为跨多个实例服务 LLM 十分常见。具体而言，全局调度器记录全局 KV\$ 缓存使用情况，并尽可能把请求调度到已经缓存相应 KV\$ 的实例。需要注意，本文工作负载感知缓存策略仅应用于单个服务实例；将其扩展到全局层留作未来工作。

评估模型与工作负载。性能评估使用三个代表性开放模

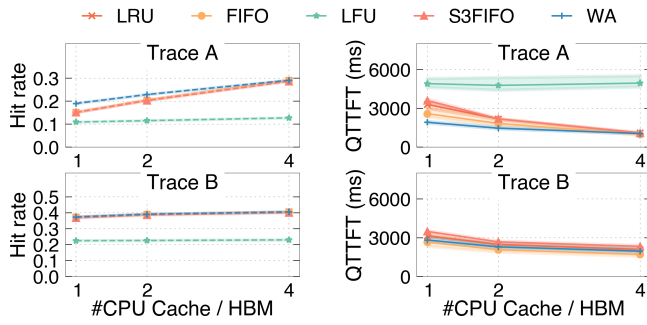


图 25: Qwen2-7B 上缓存命中率、排队首 token 时间 (QTTFT) 与所配置 CPU 缓存之间的关系。

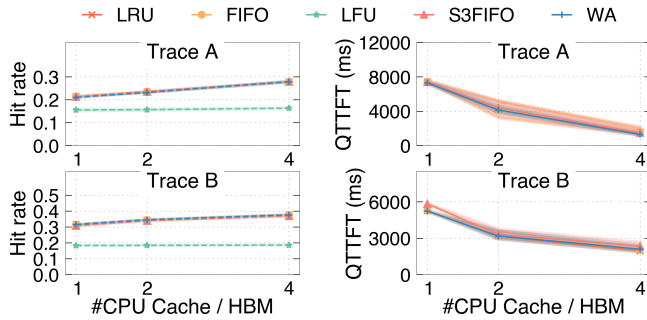


图 26: Llama2-13B 上缓存命中率、QTTFT 与所配置 CPU 缓存之间的关系。

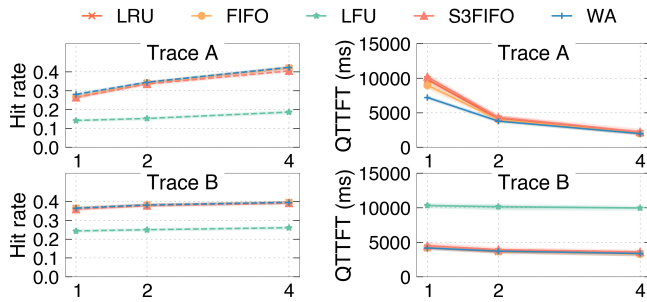


图 27: Llama3-70B 上缓存命中率、QTTFT 与所配置 CPU 缓存之间的关系。

型: Qwen2-7B [9]、Llama2-13B [53] 和 Llama3-70B [15]。选择这些模型的原因及其差异已在第 3.5 节的单请求 KV\$ 大小分析中讨论。Qwen2-7B 与 Llama2-13B 每个服务实例使用一块 GPU, Llama3-70B 使用四块 GPU。评估工作负载采用第 3.1 节描述的轨迹 A 和 B。由于 GPU 数量不足以支持真实工作负载, 本文使用保持轨迹时间模式的缩放方法 [50], 将轨迹缩放到测试平台的处理能力范围内。

匿名轨迹上的评估方法。虽然轨迹已经匿名化, 仍可通过构造精确复现原轨迹缓存命中模式的输入来评估系统服务性能。具体而言, 对每个尚未分配 token 的匿名块, 随机生成一个 token 列表, 并确保每个生成列表的哈希都不同于其他列表。为匹配原始输入的迭代次数, 还使用轨迹中的输出长度约束模型。最后, 用构造的 token 替换模型生成输出, 以保持多轮请求的缓存模式。

缓存命中率提升。图 25–27 展示了标准缓存策略与本文工

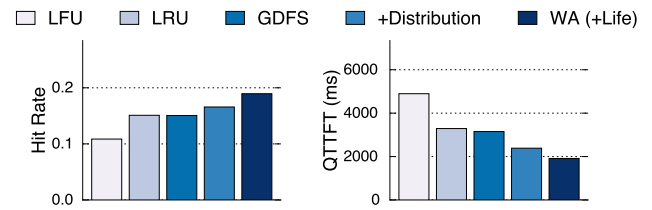


图 28: Qwen2-7B 上的消融研究, 其中 #CPU Cache / HBM 为 1。

作负载感知策略 (WA) 的命中率和服务性能。比较基线包括最近最少使用 (LRU)、先进先出 (FIFO)、最不经常使用 (LFU) 以及近期优化的 FIFO 方法 S3-FIFO [60]; 原始 GDFS 则在图 28 中比较。

由于性能取决于缓存容量, 本文通过改变所配置的 KV\$ 缓存容量来报告命中率与性能。容量相对于用于 KV\$ 的 GPU HBM 大小归一化, 即图中的“#CPU Cache / HBM”表示用于存储 KV\$ 的 CPU 内存相对于 GPU HBM 的比例。例如, 该值为 1 表示用于存储 KV\$ 的 CPU 内存大小与 GPU HBM 相等。结果带来三项观察。

第一, 与其他基线相比, 本文策略 WA 的命中率高 8.1–23.9%; 与其他基线中的最佳者相比, 高 1.5–3.9%。得益于对工作负载复用模式的了解, 本文策略更有效。第二, 缓存容量增大时, WA 的有效性会降低; 这符合预期, 因为更大缓存能够容忍较低效的策略。最后, WA 在轨迹 A 上比在轨迹 B 上更有效, 原因有二: (1) 轨迹 B 几乎没有工作负载类别信息 (> 99% 的请求都是 1 轮请求), 因此本文基于指数分布拟合的策略实质上退化为普通 LRU; (2) 轨迹 B 所需的 KV\$ 容量较小, 因此除 LFU 外, GPU HBM 已经接近最优。LFU 命中率较低, 是因为最频繁访问的块也可能很早“死亡”, 从而污染 KV\$ 缓存; 前文已对此作了详细讨论。

性能提升。缓存命中率提升会显著改善服务性能。图 25–27 还报告了服务不同轨迹时的排队 TTFT (QTTFT), 即 TTFT 加上等待 GPU 处理请求的时间。得益于 KV\$ 命中率提高, WA 将 QTTFT 降低了 28.3–41.9%。

消融研究。图 28 在轨迹 A 的 Qwen2-7B 模型上对不同策略进行消融研究; 其他模型呈现相似趋势。首先, 基于分布的方法使命中率提高 1% (即表 2 中的技术①)。使用生命周期调节器考虑 KV\$ 的短暂生命周期, 又将命中率提高 2.4% (③)。需要注意, 本文未单独描述局部性优化②, 因为它应用于所有策略。

讨论: 公平性问题。与所有现有系统及其策略一样, 恶意客户端可以发送大量具有相同长前缀的请求来独占前缀缓存, 从而造成公平性问题。保证公平性是与缓存策略彼此正交的议题, 可以通过与 FairRide [46, 51] 等 LLM 服务策略协同设计来解决。

5 相关工作

通过 KV\$ 缓存跨请求复用 KV\$。利用 KV\$ 复用加速 LLM 服务已得到广泛研究 [29, 62, 18, 19, 68, 61], 并已用于商业 LLM 服务 [41, 20, 4, 1]。目前生产系统仅把前缀匹配视为缓存命中 [62, 68, 18, 41, 20, 4, 1], 因为这样可以保持原始模型推理算法不变, 不损失准确率。一些研究 [19, 61, 22]

探索了非前缀 KV\$ 缓存方法；待这些方法成熟并部署到生产环境后，可以重新审视本文的特征刻画。

其他 KV\$ 相关优化。新近工作 [58, 67, 32, 13] 提出了运行时 KV\$ 压缩与删除方法，以减小 KV\$ 大小。StreamingLLM [58] 仅保留由最近 token 与少量初始 token 组成的有限注意力窗口。H2O [67] 推理时只纳入对注意力分数贡献最大的 token。InfiniGen [32] 推测性地选择对注意力分数至关重要的 token，并丢弃其余 token。KVQuant [21] 以很小的困惑度损失实现 3-bit KV 缓存量化，使大语言模型能够快速进行百万长度上下文推理。这些方法降低了单请求所需的 KV\$，但会带来准确率下降。本文研究与这些工作兼容：如果未压缩、完整的 KV\$ 可以复用，那么其压缩或删减版本也同样可以复用。

缓存策略优化。缓存优化长期以来一直是研究重点，包括通用缓存策略 [40, 28, 27, 52, 31, 24, 37, 70, 10, 23, 16, 11, 48, 66, 14] 与特定领域缓存 [59, 12, 49, 17]。本文延续这一研究方向，利用所刻画的 KV\$ 复用性质优化 KV\$ 缓存策略。

LLM 服务优化。本文延续提升 LLM 服务系统性能的研究 [69, 56, 30, 33, 63, 45, 18, 65]，重点刻画面向 KV\$ 缓存的服务工作负载。本文工作与 KV\$ 缓存以外的其他优化彼此正交。

6 结论

本文首次对 KV\$ 缓存系统的生产服务工作负载进行了系统、深入的刻画，并揭示了使用合成工作负载的既有研究未能捕获的若干重要发现。基于这些发现，本文进一步提出一种工作负载感知的缓存淘汰策略，改进了 LRU、LFU 等既有工作负载无关策略。我们相信，作为一个起点，本工作能够帮助研究界与工业界以工作负载驱动的方法改进当前和未来的 LLM 服务系统。

致谢

我们感谢 USENIX ATC 审稿人和论文指导人 Lei Zhang 提出的深刻反馈，衷心感谢 Guanhu Wang 协助收集轨迹数据，感谢 Xiating Xie 校对论文，感谢 Xiangfan Wu 帮助修正 token ID 哈希方法的表述，并感谢 Zhijun Yang 改进论文表述。本工作部分得到国家自然科学基金（编号 62202291、62272291、62132014）以及中央高校基本科研业务费支持；本工作还得到阿里巴巴创新研究计划提供的阿里巴巴集团科研资助。

References

- [1] Mooncake trace. https://github.com/kvcache-ai/Mooncake/blob/main/mooncake_trace.jsonl, 2025.
- [2] Sharegpt. <https://sharegpt.com>, 2025.
- [3] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023* (2023), H. Bouamor, J. Pino, and K. Bali, Eds., Association for Computational Linguistics, pp. 4895–4901.
- [4] Anthropic. Claude api. <https://www.anthropic.com/api>, 2025.
- [5] ANTHROPIC. Prompt caching. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>, 2025.
- [6] Anthropic. Prompt library. <https://docs.anthropic.com/en/prompt-library/library>, 2025.
- [7] Atikoglu, B., Xu, Y., Frachtenberg, E., Jiang, S., and Paleczny, M. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems* (2012), pp. 53–64.
- [8] Aumasson, J., and Bernstein, D. J. Siphash: A fast short-input PRF. In *Progress in Cryptology - INDOCRYPT 2012, 13th International Conference on Cryptology in India, Kolkata, India, December 9-12, 2012. Proceedings* (2012), S. D. Galbraith and M. Nandi, Eds., vol. 7668 of *Lecture Notes in Computer Science*, Springer, pp. 489–508.
- [9] Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., Hui, B., Ji, L., Li, M., Lin, J., Lin, R., Liu, D., Liu, G., Lu, C., Lu, K., Ma, J., Men, R., Ren, X., Ren, X., Tan, C., Tan, S., Tu, J., Wang, P., Wang, S., Wang, W., Wu, S., Xu, B., Xu, J., Yang, A., Yang, H., Yang, J., Yang, S., Yao, Y., Yu, B., Yuan, H., Yuan, Z., Zhang, J., Zhang, X., Zhang, Y., Zhang, Z., Zhou, C., Zhou, J., Zhou, X., and Zhu, T. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [10] Bansal, S., and Modha, D. S. CAR: clock with adaptive replacement. In *Proceedings of the FAST '04 Conference on File and Storage Technologies, March 31 - April 2, 2004, Grand Hyatt Hotel, San Francisco, California, USA* (2004), C. Thekkath, Ed., USENIX, pp. 187–200.
- [11] Beckmann, N., Chen, H., and Cidon, A. LHD: improving cache hit rate by maximizing hit density. In *15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018, Renton, WA, USA, April 9-11, 2018* (2018), S. Banerjee and S. Seshan, Eds., USENIX Association, pp. 389–403.
- [12] Berg, B., Berger, D. S., McAllister, S., Grosz, I., Gunasekar, S., Lu, J., Uhlar, M., Carrig, J., Beckmann, N., Harchol-Balter, M., and Ganger, G. R. The CacheLib caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (Nov. 2020), USENIX Association, pp. 753–768.
- [13] Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., and Xiao, W. Pyramidkv: Dynamic KV cache compression based on pyramidal information funneling. *CoRR abs/2406.02069* (2024).
- [14] Cherkasova, L. *Improving WWW proxies performance with greedy-dual-size-frequency caching policy*. Hewlett-Packard Laboratories Palo Alto, CA, USA, 1998.
- [15] Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Srivankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Rozière, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C.,

- Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I. M., Misra, I., Evtimov, I., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Upasani, K., Plawiak, K., Li, K., Heaffield, K., Stone, K., and et al. The llama 3 herd of models. *CoRR abs/2407.21783* (2024).
- [16] Einziger, G., Eytan, O., Friedman, R., and Manes, B. Lightweight robust size aware cache management. *ACM Trans. Storage* 18, 3 (2022), 27:1–27:23.
- [17] Fuerst, A., and Sharma, P. Faascache: keeping serverless computing alive with greedy-dual caching. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021* (2021), T. Sherwood, E. D. Berger, and C. Kozyrakis, Eds., ACM, pp. 386–400.
- [18] Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with cachedattention. In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), S. Bagchi and Y. Zhang, Eds., USENIX Association, pp. 111–126.
- [19] Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., and Zhong, L. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024* (2024), P. B. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., mlsys.org.
- [20] Google. Gemini api. <https://ai.google.dev/api>, 2025.
- [21] Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.
- [22] Hu, J., Huang, W., Wang, H., Wang, W., Hu, T., Zhang, Q., Feng, H., Chen, X., Shan, Y., and Xie, T. Epic: Efficient position-independent context caching for serving large language models, 2024.
- [23] Jiang, S., Chen, F., and Zhang, X. Clock-pro: An effective improvement of the CLOCK replacement. In *Proceedings of the 2005 USENIX Annual Technical Conference, April 10-15, 2005, Anaheim, CA, USA* (2005), USENIX, pp. 323–336.
- [24] Jiang, S., and Zhang, X. LIRS: an efficient low inter-reference recency set replacement policy to improve buffer cache performance. In *Proceedings of the International Conference on Measurements and Modeling of Computer Systems, SIGMETRICS 2002, June 15-19, 2002, Marina Del Rey, California, USA* (2002), R. R. Muntz, M. Martonosi, and E. de Souza e Silva, Eds., ACM, pp. 31–42.
- [25] Jin, C., Zhang, Z., Jiang, X., Liu, F., Liu, X., Liu, X., and Jin, X. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *CoRR abs/2404.12457* (2024).
- [26] Jin, S., Liu, X., Zhang, Q., and Mao, Z. M. Compute or load KV cache? why not both? *CoRR abs/2410.03065* (2024).
- [27] Johnson, T., and Shasha, D. E. 2q: A low overhead high performance buffer management replacement algorithm. In *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile* (1994), J. B. Bocca, M. Jarke, and C. Zaniolo, Eds., Morgan Kaufmann, pp. 439–450.
- [28] Karedla, R., Love, J. S., and Wherry, B. G. Caching strategies to improve disk system performance. *Computer* 27, 3 (1994), 38–46.
- [29] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023* (2023), J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds., ACM, pp. 611–626.
- [30] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023* (2023), J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds., ACM, pp. 611–626.
- [31] Lee, D., Choi, J., Kim, J., Noh, S. H., Min, S. L., Cho, Y., and Kim, C. LRFU: A spectrum of policies that subsumes the least recently used and least frequently used policies. *IEEE Trans. Computers* 50, 12 (2001), 1352–1361.
- [32] Lee, W., Lee, J., Seo, J., and Sim, J. Infinigen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), A. Gavrilovska and D. B. Terry, Eds., USENIX Association, pp. 155–172.
- [33] Li, Z., Zheng, L., Zhong, Y., Liu, V., Sheng, Y., Jin, X., Huang, Y., Chen, Z., Zhang, H., Gonzalez, J. E., and Stoica, I. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)* (Boston, MA, July 2023), USENIX Association, pp. 663–679.
- [34] Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of llm-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), A. Gavrilovska and D. B. Terry, Eds., USENIX Association, pp. 929–945.
- [35] Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

- [36] Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., and Jiang, J. Cachegen: KV cache compression and streaming for fast large language model serving. In Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024 (2024), ACM, pp. 38–56.
- [37] Megiddo, N., and Modha, D. S. ARC: A self-tuning, low overhead replacement cache. In Proceedings of the FAST '03 Conference on File and Storage Technologies, March 31 - April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA (2003), J. Chase, Ed., USENIX.
- [38] Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (2021), pp. 1–15.
- [39] Nishtala, R., Fugal, H., Grimm, S., Kwiatkowski, M., Lee, H., Li, H. C., McElroy, R., Paleczny, M., Peek, D., Saab, P., Stafford, D., Tung, T., and Venkataramani, V. Scaling memcache at facebook. In 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13) (Lombard, IL, Apr. 2013), USENIX Association, pp. 385–398.
- [40] O’Neil, E. J., O’Neil, P. E., and Weikum, G. The LRU-K page replacement algorithm for database disk buffering. In Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, Washington, DC, USA, May 26-28, 1993 (1993), P. Buneman and S. Jajodia, Eds., ACM Press, pp. 297–306.
- [41] OpenAI. Openai developer platform. <https://platform.openai.com/docs/overview>.
- [42] OpenAI. Chatgpt. <https://chatgpt.com>, 2024.
- [43] OpenAI. Introducing openai o1. <https://openai.com/o1/>, 2025.
- [44] OpenAI. Prompt caching. <https://platform.openai.com/docs/guides/prompt-caching>, 2025.
- [45] Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In 51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024 (2024), IEEE, pp. 118–132.
- [46] Pu, Q., Li, H., Zaharia, M., Ghodsi, A., and Stoica, I. Fairride: Near-optimal, fair cache sharing. In 13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16-18, 2016 (2016), K. J. Argyraki and R. Isaacs, Eds., USENIX Association, pp. 393–406.
- [47] Rock, D. Your brain at work: Strategies for overcoming distraction, regaining focus, and working smarter all day long. Journal of Behavioral Optometry 21, 5 (2010), 130.
- [48] Rodriguez, L. V., Yusuf, F. B., Lyons, S., Paz, E., Rangaswami, R., Liu, J., Zhao, M., and Narasimhan, G. Learning cache replacement with CACHEUS. In 19th USENIX Conference on File and Storage Technologies, FAST 2021, February 23-25, 2021 (2021), M. K. Aguilera and G. Yadgar, Eds., USENIX Association, pp. 341–354.
- [49] Roy, R. B., Patel, T., and Tiwari, D. Icebreaker: warming serverless functions better with heterogeneity. In ASPLOS ’22: 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, 28 February 2022 - 4 March 2022 (2022), B. Falsafi, M. Ferdman, S. Lu, and T. F. Wenisch, Eds., ACM, pp. 753–767.
- [50] Sajal, S. M., Hasan, R., Zhu, T., Urgaonkar, B., and Sen, S. Tracesplitter: a new paradigm for downscaling traces. In EuroSys ’21: Sixteenth European Conference on Computer Systems, Online Event, United Kingdom, April 26-28, 2021 (2021), A. Barbalace, P. Bhatotia, L. Alvisi, and C. Cadar, Eds., ACM, pp. 606–619.
- [51] Sheng, Y., Cao, S., Li, D., Zhu, B., Li, Z., Zhuo, D., Gonzalez, J. E., and Stoica, I. Fairness in serving large language models. In 18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024 (2024), A. Gavrilovska and D. B. Terry, Eds., USENIX Association, pp. 965–988.
- [52] Smaragdakis, Y., Kaplan, S. F., and Wilson, P. R. EELRU: simple and effective adaptive page replacement. In Proceedings of the 1999 ACM SIGMETRICS international conference on Measurement and modeling of computer systems, Atlanta, Georgia, USA, May 1-4, 1999 (1999), D. A. Menascé and C. Williamson, Eds., ACM, pp. 122–133.
- [53] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. CoRR abs/2302.13971 (2023).
- [54] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA (2017), I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., pp. 5998–6008.
- [55] Wang, X., Wang, Z., Liu, J., Chen, Y., Yuan, L., Peng, H., and Ji, H. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. arXiv preprint arXiv:2309.10691 (2023).
- [56] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., and Jin, X. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. CoRR abs/2404.09526 (2024).
- [57] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In First Conference on Language Modeling (2024).
- [58] Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024 (2024), OpenReview.net.
- [59] Yang, J., Yue, Y., and Rashmi, K. V. A large scale analysis of hundreds of in-memory cache clusters at twitter. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20) (Nov. 2020), USENIX Association, pp. 191–208.

- [60] Yang, J., Zhang, Y., Qiu, Z., Yue, Y., and Vinayak, R. Fifo queues are all you need for cache eviction. In Proceedings of the 29th Symposium on Operating Systems Principles (New York, NY, USA, 2023), SOSP '23, Association for Computing Machinery, p. 130–149.
- [61] Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. Cacheblend: Fast large language model serving for RAG with cached knowledge fusion. CoRR abs/2405.16444 (2024).
- [62] Ye, L., Tao, Z., Huang, Y., and Li, Y. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (2024), Association for Computational Linguistics, pp. 11608–11620.
- [63] Yu, G., Jeong, J. S., Kim, G., Kim, S., and Chun, B. Orca: A distributed serving system for transformer-based generative models. In 16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022 (2022), M. K. Aguilera and H. Weatherspoon, Eds., USENIX Association, pp. 521–538.
- [64] Yu, L., and Li, J. Stateful large language model serving with pensieve. CoRR abs/2312.05516 (2023).
- [65] Zhang, D., Wang, H., Liu, Y., Wei, X., Shan, Y., Chen, R., and Chen, H. Fast and live model auto scaling with O(1) host caching. CoRR abs/2412.17246 (2024).
- [66] Zhang, Y., Yang, J., Yue, Y., Vigfusson, Y., and Rashmi, K. V. SIEVE is simpler than LRU: an efficient turn-key eviction algorithm for web caches. In 21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024 (2024), L. Vanbever and I. Zhang, Eds., USENIX Association.
- [67] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C. W., Wang, Z., and Chen, B. H2O: heavy-hitter oracle for efficient generative inference of large language models. In Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023 (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.
- [68] Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. Efficiently programming large language models using sglang. CoRR abs/2312.07104 (2023).
- [69] Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In 18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024 (2024), A. Gavrilovska and D. B. Terry, Eds., USENIX Association, pp. 193–210.
- [70] Zhou, Y., Chen, Z., and Li, K. Second-level buffer cache management. IEEE Trans. Parallel Distributed Syst. 15, 6 (2004), 505–519.

A 附录

A.1 不同日期的轨迹结果

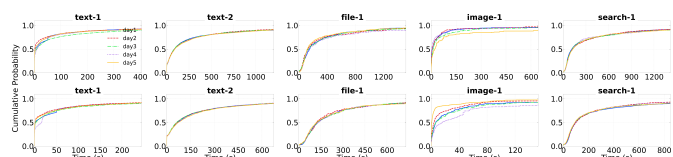


图 29: 5 个工作日内复用概率累积分布函数的演化。第一行: 上午 9:00 至 10:00 的分布; 第二行: 上午 10:00 至 11:00 的分布。

图 29 展示了连续 5 个工作日内, 各请求类别在未来时间窗口内的复用概率累积分布函数 (CDF) 如何演化。可以观察到, 工作日上午 9:00 至 10:00 的复用概率彼此相似。因此, 可以离线分析前一天的复用日志, 预测次日各类别的复用概率。不过, 图像类别的复用概率比其他类别更难预测; 作者推测, 这是因为图像类别的可复用粒度比文本类别更粗, 而且不同用户之间的差异更大。